

University of Puerto Rico
Mayagüez Campus
Mayagüez, Puerto Rico

Department of Electrical and Computer Engineering

Programming Assignment 4 - Beyond Classical Search: Search

by

Diego H. Merchan Rueda
Marco A. Monserrat Montoto
Edgard A. Díaz Bobé
Malik J. Paramo Rios

For: Professor Jose F. Vega Riveros
Course: ICOM5015, Section 001D
Date: April 28, 2026

Introduction	3
Purpose of Assignment	3
Hypothesis	3
Experimental Design Choice	4
Methodology	4
Algorithms	5
Performance	5
Results and Analysis	5
Conclusions	7
Lessons Learned	7
References	8
Group Collaboration	8

Introduction

Purpose of Assignment

The purpose of this assignment is to complete Exercise 4.4 from the AIMA exercises by using the UC Berkeley Python code repository to study local search algorithms experimentally [1, 2]. Instead of writing all the algorithms completely from scratch, the focus is on testing their behavior, collecting results, and analyzing their performance. This assignment compares steepest-ascent hill climbing, first-choice hill climbing, random-restart hill climbing, and simulated annealing on the 8-puzzle and 8-queens problems as it can be seen in Figure 1. The results are organized using tables and figures so the performance of each algorithm can be evaluated clearly. The main measures used in the comparison are the percentage of problems solved and the search cost required by each method. This allows the report to discuss not only which algorithms solve more instances, but also how much effort they require to reach those solutions.

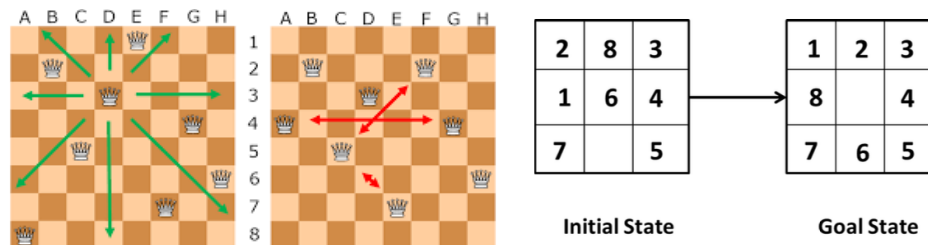


Figure 1. Example of 8-puzzle and 8-queens problems

Hypothesis

It is expected that the local search algorithms will perform differently depending on the structure of the problem. For the 8-queens problem, random-restart hill climbing and simulated annealing are expected to solve a higher percentage of instances because they are better suited for escaping local maxima and poor initial configurations. Steepest-ascent and first-choice hill climbing are expected to have lower search cost, but they may fail more often because they can stop when no immediate improvement is available. For the 8-puzzle problem, it is expected that all local search algorithms will have more difficulty reaching the goal because improving the heuristic value does not always guarantee progress toward the correct solution path. Therefore, simulated annealing and random restart may perform better than simple hill climbing, but they may also require a higher search cost. Overall, the experiment is expected to show a tradeoff between reliability and search effort.

Experimental Design Choice

The experimental design uses a Jupyter Notebook so the code, results, tables, figures, and written analysis can be presented together in one organized report. The implementation is based on the UC Berkeley AIMA Python repository, following the assignment instructions. Additional code is used mainly to generate random problem instances, collect measurements, and summarize the results. The experiment compares four local search algorithms: steepest-ascent hill climbing, first-choice hill climbing, random-restart hill climbing, and simulated annealing. Each algorithm is tested on multiple randomly generated instances of the 8-puzzle and the 8-queens problem. For each run, the program records whether the algorithm solved the instance and the search cost required to do so. In this report, search cost is measured as the number of neighbor or candidate states evaluated during the search. For the 8-puzzle, random solvable states are generated by starting from the goal configuration and applying a sequence of legal random moves. For the 8-queens problem, random complete board configurations are generated with one queen placed in each column. After collecting the results, the data is grouped by problem and algorithm to calculate the percentage of solved instances and the average search cost. These values are then presented in tables and figures so the performance of each algorithm can be compared clearly.

Methodology

For the experiment design, two AI benchmark problems were selected to evaluate the local search algorithm: the 8-Queens problem and the 8-Puzzle problem. For 50 independent trials, one random instance of each problem was generated. The 8-Queens instances were initialized by assigning each queen to a uniformly random row in its column, while the 8-Puzzle instances were generated by applying 20 random legal moves starting from the goal state to ensure solvability. All four algorithms were evaluated on the same instance within each trial, yielding a total of 400 experimental runs.

The 8-Queen problem was modeled as a complete-state formulation where a state is a tuple of length 8, each element representing the row of the queen in that column. The evaluation function returns the negative number of attacking queen pairs, so the maximum corresponds to a valid solution with zero conflicts. The 8-Puzzle problem is extended with a value function based on the negative Manhattan distance between the current state and the goal[3].

Algorithms

- 1) **Steepest-Ascent Hill Climbing:** At each step, all neighboring states reachable by a single action are generated and evaluated. The algorithm moves to the neighbor with the highest value. This terminates when no neighbor strictly improves the current value or when the goal is reached.
- 2) **First-Choice Hill Climbing:** Neighbors are generated in a random order and the algorithm moves to the first neighbor that improves the current value. If no improving value is found, the algorithm terminates.
- 3) **Random-Restart Hill Climbing:** The Steepest-ascent algorithm is executed repeatedly from independently sampled random initial states. This procedure terminates as soon as a solution is found, or after a fixed number of restarts is reached. The search cost for this is the cumulative total across all restarts.
- 4) **Simulated Annealing:** A random neighbor is selected at each step. Worsening moves are accepted with a probability that depends on the magnitude of the deterioration and current temperature. Temperature is reduced exponentially over time using an initial value of 20 and a cooling rate of 1. The algorithm terminates when the goal is reached, or the step limit of 10,000 is exceeded[4].

Performance

The performance followed two metrics: Percentage of solved instances and search cost. The percentage of solved instances is the proportion of trials in which the algorithm reached the goal state, expressed as a percentage. The search cost is the total number of candidate states evaluated.

Results and Analysis

According to the results observed, 3 of the 4 algorithms performed well depending on the complexity level of the problem. The results obtained include the optimal solution for the given problem, whether it was solved, the search cost, and the initial and final states. This information was recorded for each algorithm across the 50 trials performed for each problem. In this way, the table shown in Table 1 was created, summarizing the performance of each algorithm for each type of problem.

problem	algorithm	percentage solved	average search cost	median search cost	number of trials
8-queens	RRHC	100%	1528.80	1344	50
8-queens	SA	86%	1240.92	1100	50

8-queens	SAHC	20%	211.68	224	50
8-queens	FCHC	12%	93.34	87	50
8-puzzles	RRHC	100%	17.44	17	50
8-puzzles	SAHC	68%	13.46	15	50
8-puzzles	FCHC	62%	8.74	8.0	50
8-puzzles	SA	16%	1669.54	1976.0	50

Table 1. Overall Summary

According to what can be observed here, only one model works perfectly for any level of complexity: Random-Restart Hill Climbing. This is because this algorithm keeps searching for a solution based on the number of restarts we assign. In this case, the 30 restarts were enough for the 50 trials of each problem. However, what happens with the complexity of the problems and the other algorithms? In this experiment, we considered 8-queens as a more complex problem and 8-puzzle as a simpler problem. The two hill climbing algorithms, First-Choice Hill Climbing and Steepest-Ascent Hill Climbing, are basic search algorithms that work better for simpler problems. For that reason, they performed acceptably on the 8-puzzle problem. On the other hand, Simulated Annealing performed better on the more complex problem because its logic allows it to escape from local optima by sometimes accepting worse moves. This behavior can be observed more clearly in the following graphs.

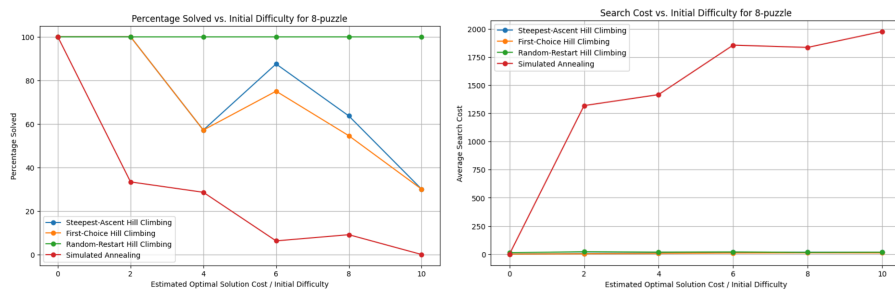


Figure 2. 8-Puzzle Percentage Solved and Search Cost vs Optimal Solution

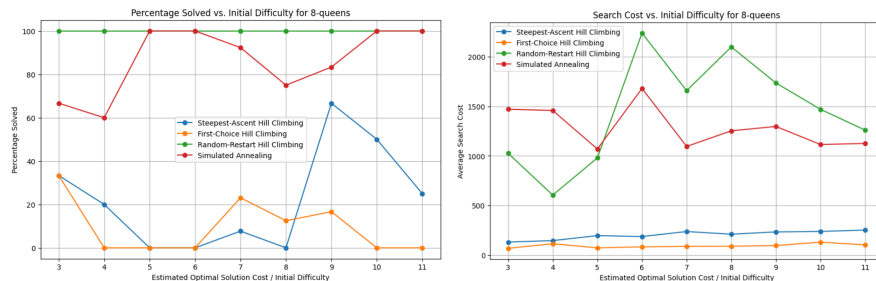


Figure 3. 8-Queens Percentage Solved and Search Cost vs Optimal Solution

For the graphs, Random-Restart Hill Climbing is green, First-Choice Hill Climbing is yellow/orange, Steepest-Ascent Hill Climbing is blue, and Simulated Annealing is red. In Figure 2, Random-Restart stays at the highest success rate, while Simulated Annealing performs poorly on the 8-puzzle because this problem does not require as much exploration. This is also shown in the search cost graph, where Simulated Annealing has a much higher cost than the other algorithms. However, for the more complex 8-queens problem in Figure 3, Simulated Annealing becomes more useful because it explores more options and reaches a success rate above 80%. Overall, Random-Restart Hill Climbing was the most reliable algorithm across both problem types.

Conclusions

In this assignment four local search algorithms are implemented which were steepest ascent hill climbing, first choice hill climbing, hill climbing with random restart, and the simulated annealing. They were applied to the 8 puzzle and 8 queen problems using UC Berkeley repository as reference. The results showed that no algorithm is objectively better than their counterparts when presented with multiple different conditions. The steepest ascent hill climbing is fast, but gets stuck a lot in local optima, especially in higher difficulties. Random restarts made big improvements by addressing weakness effectively leading to higher solution rates through diversification in searching, but it required more total state evaluations. First choice hill climbing also gets trapped like steepest, but evaluates less states per step. Simulated annealing escaped local optima without full restarts by focusing on balanced exploration early and exploiting later. As the difficulty rose, solution rates lowered and search cost went up, but rate of degradation always differed greatly between algorithms.

Lessons Learned

1. Every algorithm had its pros and cons, needing ample testing to quantify each behavior for measuring performance.
2. The value function was a major deciding factor, and poorly chosen leads to randomized behavior.
3. Local optima have the biggest challenge with some algorithms getting stuck in this stage.
4. More restarts and randomness improved how reliable the algorithm was but also came with added cost.

AI Disclaimer:

In accordance with academic integrity guidelines, the authors disclose that artificial intelligence tools were used in the preparation of this assignment. Specifically, Claude (Anthropic) was used to assist in reviewing and debugging the Python notebook, verifying the correctness of the algorithm implementations against the UC Berkeley AIMA repository, and the AI disclaimer. All experimental design decisions, code logic, analysis of results, and interpretation of algorithm behavior were carried out by the authors. The AI tool was used strictly as a supplementary aid and not as a substitute for the authors' own understanding and judgment. All content has been reviewed, verified, and approved by the authors prior to submission.

References

[1] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[2] aimacode, “Artificial Intelligence: A Modern Approach Code Repository.” [Online]. Available: <https://github.com/aimacode>. [Accessed: Apr. 22, 2026].

[3] GeeksforGeeks, “Manhattan Distance.” Available: <https://www.geeksforgeeks.org/data-science/manhattan-distance/>. [Accessed: Apr. 27, 2026].

[4] The Knowledge Academy, “Simulated Annealing: Everything You Should Know,” The Knowledge Academy, 2024. Accessed: Apr. 28, 2026. [Online]. Available: <https://www.theknowledgeacademy.com/blog/simulated-annealing/>

Group Collaboration

1. Diego - Analysis / Code
2. Malik - Methodology/ Results
3. Edgard- Introduction / Experimental Design Choice
4. Marco - Conclusion / Lessons Learned

Video Link: <https://youtu.be/bhjL7jngUjo>

README:

- The project should have the following folder structure:
 - HW4_Group/aima-python/[search.py](#)

- code_groupM/ HW4_Exercise_4_4_Local_Search.ipynb

- The requirements are:

absl-py==2.4.0
anyio==4.13.0
argon2-cffi==25.1.0
argon2-cffi-bindings==25.1.0
arrow==1.4.0
asgiref==3.11.1
asttokens==3.0.1
astunparse==1.6.3
async-lru==2.3.0
attrs==26.1.0
babel==2.18.0
beautifulsoup4==4.14.3
bleach==6.3.0
certifi==2026.4.22
cffi==2.0.0
charset-normalizer==3.4.7
colorama==0.4.6
comm==0.2.3
contourpy==1.3.2
coverage==7.13.5
cvxopt==1.3.3
cyclers==0.12.1
debugpy==1.8.20
decorator==5.2.1
defusedxml==0.7.1
Django==5.2.13
exceptiongroup==1.3.1
executing==2.2.1
fastjsonschema==2.21.2
flatbuffers==25.12.19
fonttools==4.62.1
fqdn==1.5.1
gast==0.7.0
google-pasta==0.2.0
grpcio==1.80.0
h11==0.16.0
h5py==3.14.0

httpcore==1.0.9
httpx==0.28.1
idna==3.13
image==1.5.33
iniconfig==2.3.0
ipykernel==7.2.0
ipython==8.39.0
ipythonblocks==1.9.1
ipywidgets==8.1.8
isoduration==20.11.0
jedi==0.19.2
Jinja2==3.1.6
json5==0.14.0
jsonpointer==3.1.1
jsonschema==4.26.0
jsonschema-specifications==2025.9.1
jupyter==1.1.1
jupyter-console==6.6.3
jupyter-events==0.12.1
jupyter-lsp==2.3.1
jupyter_client==8.8.0
jupyter_core==5.9.1
jupyter_server==2.17.0
jupyter_server_terminals==0.5.4
jupyterlab==4.5.6
jupyterlab_pygments==0.3.0
jupyterlab_server==2.28.0
jupyterlab_widgets==3.0.16
keras==3.12.1
kiwisolver==1.5.0
lark==1.3.1
libclang==18.1.1
markdown-it-py==4.0.0
MarkupSafe==3.0.3
matplotlib==3.10.9
matplotlib-inline==0.2.1
mdurl==0.1.2
mistune==3.2.0
ml_dtypes==0.5.4
namex==0.1.0

nbclient==0.10.4
nbconvert==7.17.1
nbformat==5.10.4
nest-asyncio==1.6.0
networkx==3.4.2
notebook==7.5.5
notebook_shim==0.2.4
numpy==2.2.6
opencv-python==4.13.0.92
opt_einsum==3.4.0
optree==0.19.0
overrides==7.7.0
packaging @
file:///home/conda/feedstock_root/build_artifacts/bld/rattler-build_packaging_177710362
1/work
pandas==2.3.3
pandocfilters==1.5.1
parso==0.8.6
pillow==12.2.0
platformdirs==4.9.6
pluggy==1.6.0
prometheus_client==0.25.0
prompt_toolkit==3.0.52
protobuf==7.34.1
psutil==7.2.2
pure_eval==0.2.3
pyparser==3.0
Pygments==2.20.0
pyparsing==3.3.2
pytest==9.0.3
pytest-cov==7.1.0
python-dateutil==2.9.0.post0
python-json-logger==4.1.0
pytz==2026.1.post1
pywinpty==3.0.3
PyYAML==6.0.3
pyzmq==27.1.0
qpsolvers==4.11.0
referencing==0.37.0
requests==2.33.1

rfc3339-validator==0.1.4
rfc3986-validator==0.1.1
rfc3987-syntax==1.1.0
rich==15.0.0
rpds-py==0.30.0
scipy==1.15.3
Send2Trash==2.1.0
six==1.17.0
sortedcontainers==2.4.0
soupsieve==2.8.3
sqlparse==0.5.5
stack-data==0.6.3
tensorflow==2.21.0
termcolor==3.3.0
terminado==0.18.1
tinycss2==1.4.0
tomli==2.4.1
tornado==6.5.5
traitlets==5.14.3
typing_extensions==4.15.0
tzdata==2026.2
uri-template==1.3.0
urllib3==2.6.3
wcwidth==0.6.0
webcolors==25.10.0
webencodings==0.5.1
websocket-client==1.9.0
widgetsnbextension==4.0.15
wrapt==2.1.2